Compiling Facts into Applications

Samuel Lippert
Drivly Inc

Abstract

Domain knowledge enters software as natural language requirements, is re-encoded as schemas, validation logic, state machines, and API routes, and drifts out of sync in production. The informal layer between specification and execution is where bugs live and where automated agents hallucinate. We show that a single set of FORML 2 declarations can define a database schema, constraint rules, state machine workflows, and a REST API with hypermedia navigation, producing a complete application from domain facts alone. We formalize this as AREST, a Backus AST system [1] whose definitions are compiled FORML 2 readings [5] and whose state is Codd's named set [2] of ORM 2 elementary facts. REST endpoints are applications of a single system function. Ingesting new readings is self-modification. Five theorems establish that the grammar is unambiguous, that specifications and executables are the same object, that state transfer is complete, that HATEOAS links (both transitions and entity hierarchy) are projections of the population, and that every value in the API is derived from facts. Together they eliminate the informal layer entirely.

1 Introduction

Software begins as domain knowledge in someone's head. It is re-encoded as schemas, validation logic, state machines, and API routes. When these are distinct artifacts, their equivalence is not preserved by construction.

AREST identifies what was never separate to begin with. A FORML 2 reading is simultaneously a relation schema, a constraint specification, a resource definition, and an FFP object. The compiler does not translate between representations. It recognizes a single object that already occupies all four roles.

2 Terminology

Three terms carry overloaded abbreviations. *AST* follows Backus's usage (Applicative State Transition, 1978), not the compiler term. *ORM* follows Halpin's usage (Object-Role Modeling, 2008), not the persistence pattern. *FP* follows Backus's usage (Function-level Programming), not the broader tradition. *FFP* is Backus's extension of FP with named definitions and a representation function ρ that maps objects to the functions they denote [1, Sec. 13].

3 Definitions

Each source formalism solved one layer: Backus the evaluation model, Codd the data model, Halpin the constraint language, and Fielding the transfer protocol. The contribution of AREST is identification, not integration. A FORML 2 reading is simultaneously an FFP object, a relation schema, a constraint specification, and a resource definition, and the compiler recognizes this identity. The five theorems verify it.

Definition 1 (AREST)**.** An *Applicative REpresentational State Transfer* system is an AST system [1, Sec. 14] whose state D satisfies:

- The *FILE* cell contains a population $P = \uparrow \mathit{FILE} : D$, a named set of relations [2, Sec. 1.6] whose elements are ORM 2 elementary facts [5, Ch. 2]
- The *DEFS* cells contain FFP objects [1, Sec. 13.2]. These are readings compiled from FORML 2 declarations [5], and functions registered by the runtime

The system function $\mathit{SYSTEM} : x = \langle o, D' \rangle$ [1, Sec. 14.3.1] is defined by the definitions in D. REST endpoints are applications of *SYSTEM* with distinct inputs. The representation function ρ [1, Sec. 13.3.2] maps an object to the function it represents. Given a fact type, ρ looks it up in *DEFS* and returns the functional form that determines what happens when an operation is applied. The universe of FFP objects O and ρ are inherited from the AST framework.

A fact type [5, Ch. 2] is a constructor (*CONS*) of roles [1, Sec. 11.2.4]. Populate the roles with object instances and you get a fact. The fact is a sequence $\langle \mathit{ft}, o_1, \ldots, o_n \rangle$ where *ft* is the fact type and $o_1, \ldots, o_n$ are the objects playing its roles. Metacomposition [1, Sec. 13.3.2] resolves the fact by looking up its fact type:

$$(\rho \langle x_1, \ldots, x_n \rangle) : y = (\rho\, x_1) : \langle \langle x_1, \ldots, x_n \rangle, y \rangle \qquad (1)$$

ρ resolves the fact type (x_1) from *DEFS* to determine the functional form, and the full fact ($\langle x_1, \ldots, x_n \rangle$) and the operand ($y$) are passed to it. The operand is the operation (*render*, *httpFetch*, *validate*, or any function the runtime has registered). The fact is a function in the λ-calculus sense: $\lambda f.\, f(o_1, \ldots, o_n)$, accepting an operation and applying it to its own objects.

4 Operations

4.1 Domain Primitives

Each combining form [1, Sec. 11.2.4] is an FFP object with a domain interpretation:

FFP Object	Form	Domain
$\langle CONS, s_1, \ldots, s_n \rangle$	Construction	Fact Type
s_i	Selector	Role
$\langle COMP, f, g \rangle$	Composition	Derivation
$\langle COND, p, f, g \rangle$	Condition	Guard
$\langle \alpha, f \rangle$	Apply-to-all	Traversal
$\langle /, f \rangle$	Insert	Aggregation
$\langle bu, f, x \rangle$	Binary-to-unary	Query
$\bar{x}$	Constant	Literal
distl	Distribute-left	Join preparation
eq	Equality test	Predicate

Table 1: FFP primitives with domain interpretations.

A fact type $\langle CONS, s_1, \ldots, s_n \rangle$ (Table 1) is a function that produces a fact when applied to object instances. Partially applied via *bu* (Table 1) it produces a query.

Two derived forms used throughout the paper:

$$Filter(p) \equiv \alpha\,(p \to id;\ \bot) \tag{2}$$

Filter is α (apply-to-all) of a condition: keep elements where p holds, replace others with $\bot$.

$$\begin{aligned} foldl\ f\ z\ \langle\rangle &= z \\ foldl\ f\ z\ \langle e, E' \rangle &= foldl\ f\ (f\ z\ e)\ E' \end{aligned} \tag{3}$$

foldl is a left fold derivable from Backus's *while* [1, Sec. 11.2.4]: $foldl\ f\ z\ E \equiv (while\ not \circ null \circ 1\ \ [f \circ [2, 1 \circ 1],\ tl \circ 1]) \circ 2 : \langle E, z \rangle$, where 1 and 2 select the event stream and accumulator from the pair. The right fold $/f$ [1, Sec. 11.2.4] processes right to left. State machines require left to right (chronological event order), hence *foldl*.

The compiled form for each FORML 2 construct:

FORML 2 Construct	Compiled FFP Object
Entity type `X(.ref)`	$\langle CONS, s_1, \ldots, s_n \rangle$
Fact type `X r Y`	$\langle CONS, s_1, s_2 \rangle$ (binary)
Derivation rule	$\langle COMP, f, g \rangle$
State machine	*foldl transition* $s_0\ E$ (3)
Instance fact	$\bar{x}$ asserted into P
Constraints (all compile to $Filter(p) : P$ *(2))*	
Cardinality family: $p = count(scope) \notin [k, m]$	
UC, MC, FC, XC, XO, OR	scope $\in$ {role, participation}
Membership family: $p = match(target, polarity)$	
IR, AS, SY, AT (self-ref)	target = same population, reversed
IT, TR, AC (path)	target = transitive closure step
SS, EQ (set comparison)	target = cross-fact-type tuples
VC (value)	target = declared value set

Every ORM 2 constraint compiles to a restriction (6) whose predicate belongs to one of two families. The *cardinality* family checks whether a count falls within bounds $[k, m]$: UC sets $[0, 1]$, MC sets $[1, \infty)$, FC sets $[k, m]$. The *membership* family checks whether a tuple exists (or is absent) in a target set. Ring constraints match against the same population with reversed or transitively extended roles. Subset and equality constraints match across distinct fact types. Value constraints match against a declared constant set. Polarity (must exist vs. must not exist) is the only parameter distinguishing symmetric (SY, violation when reverse is absent) from asymmetric (AS, violation when reverse is present).

4.2 Relational Algebra

Codd's adequate θ_1 [2, Sec. 2.2] as FFP definitions (primitives from Table 1, *Filter* from (2)):

$$\pi_L(R) \equiv \alpha\,[s_{i_1}, \ldots, s_{i_k}] : R \tag{4}$$

$$R * S \equiv Filter(eq \circ [s_{sh}]) \circ distl \tag{5}$$

$$R_{L|_M} S \equiv Filter(p) : R \tag{6}$$

$$R \cdot S \equiv \pi_{1s}(R * S) \tag{7}$$

$$\gamma(R) \equiv Filter(eq \circ [s_1, s_n]) : R \tag{8}$$

where s_{sh} selects the roles shared between R and S.

P is a named set of relations in normal form (ORM 2 elementary facts are 5NF by construction [5, Ch. 12]). Codd shows that $\theta_1 = \{$projection, natural join, restriction, tie$\}$ is adequate for the named set [3]. Equations (4)–(8) express each element of θ_1 as an FFP definition. The combining forms that query the population are the same combining forms that evaluate constraints and derive facts. There is no separate query language. The evaluation algebra is the query algebra.

4.3 State and Cells

The state D is a sequence of cells $\langle CELL, n, c \rangle$ [1, Sec. 14.3]:

$$\uparrow n : D \to c \qquad \text{(fetch)} \tag{9}$$

$$\downarrow n : \langle x, D \rangle \to D' \qquad \text{(store)} \tag{10}$$

FILE is a cell whose contents is P. RMAP [5, Ch. 17] determines which facts belong to which cell from the schema's uniqueness constraints. The result is a 3NF row: the complete set of facts that depend on one entity's key. Each entity is a cell. The cell stores exactly the facts that RMAP groups under that entity. A cell is what it says it is: a complete object of a type. Cells may contain sub-stores [1, Sec. 14.7]. Partitioning is a cell containing a subset of P.

5 The System Function

Each input x addresses an entity and carries an operation. The runtime maps the HTTP method to a registered function (`GET` $\rightarrow$ *read*, `POST` $\rightarrow$ *create*, `PUT` $\rightarrow$ *update*, `DELETE` $\rightarrow$ *delete*). The system function routes the input to the addressed entity and passes the operation as the operand:

$$SYSTEM : x = (\rho\,(\uparrow entity(x) : D)) : \uparrow op(x) \tag{11}$$

The entity is fetched from D. ρ resolves it to a functional form determined by its fact type and the definitions in *DEFS*. The operation (*read*, *create*, etc.) is applied as the operand. The entity handles the dispatch, not the system function. In conventional architectures, a router dispatches operations to handlers. Adding a new operation or a new entity type requires changing the router. Here, the entity receives an operation it has never seen before and applies it, because the entity is a λ-function and the operation is its argument. New operations are registered in *DEFS* without modifying any entity, and new entity types are added without modifying any operation. This is what makes the system shardable: each entity is self-contained, and two entities on different machines apply the same operation without coordinating.

Self-modification is the same mechanism. Ingesting readings [1, Sec. 14.6] is an application of *SYSTEM* where the addressed entity is *DEFS* and the operation is $compile \circ parse$. The new FFP objects are stored via $\downarrow DEFS$. Subsequent applications of *SYSTEM* evaluate the new constraints and derivation rules. The system extends its own program.

5.1 Commands

μ denotes evaluation [1, Sec. 14.3.1]. Applying *SYSTEM* to an input produces an output o and a new state D'. Each application is an AST transition: the state before, the state after, nothing else changes.

A Create command:

$$create = emit \circ validate \circ derive \circ resolve \tag{12}$$

resolve determines identity from the reference scheme. *derive* forward-chains derivation rules to the least fixed point. *validate* applies every constraint as a restriction (6) over the complete population, producing violation sets. *emit* always runs. An alethic violation produces a rejection and the state is unchanged ($D' = D$). A deontic violation produces warnings but the operation succeeds ($D' = D''$). The violation message is the original reading (Corollary 2).

A state machine is a left fold over an event stream:

$$machine(s_0, E) = foldl\ transition\ s_0\ E \tag{13}$$

$$transition(s, e) = (p \rightarrow \bar{s'};\ \bar{s}) : \langle s, e \rangle \tag{14}$$

where $foldl\ f\ z\ \langle\rangle = z$ and $foldl\ f\ z\ \langle e, E' \rangle = foldl\ f\ (f\ z\ e)\ E'$. The transition function is a chain of Backus conditions [1, Sec. 11.2.4]: $(p \rightarrow f;\ g)$ where p tests whether $\langle s, e \rangle$ matches a declared transition, f returns the target state, and g tries the next condition or returns the current state $\bar{s}$ as fallback. Events are stored and processed in chronological order. The fold is pure replay. Applying the event stream to a machine produces the current state without side effects, and no entry verbs or callbacks execute. This is restoration of a hibernating machine. Only a new event, not yet in the stream, causes a real transition where the entry verb fires and effects are produced.

Deletion is a transition to a terminal state, not a distinct primitive. The entity receives *delete* as the operation. It reaches a state with no outgoing transitions and is excluded from query results by restriction (6).

5.2 Platform Binding

The runtime registers functions into *DEFS* via $\downarrow DEFS$. A browser runtime registers rendering functions, and a server runtime registers *httpFetch*, *upsert*, and *notify*. In practice, metacomposition (1) looks like this:

$$(\rho\, fact) : render \rightarrow widget$$
$$(\rho\, fact) : httpFetch \rightarrow response$$
$$(\rho\, fact) : upsert \rightarrow D'$$

The fact receives a function and applies it to its own objects. Runtime functions that interact with external state execute during *resolve* (12), populating cells before *derive* runs, and functions that consume the representation execute after *emit*. This keeps *derive* pure, monotonic, and terminating over P. A single *SYSTEM* serves browser, server, and storage runtimes by varying *DEFS*, not by varying the logic. Because the binding is via ρ, it is live: when a cell's contents change via $\downarrow$, the ρ-application re-evaluates and the bound function fires, making event streaming the re-evaluation of ρ over updated D rather than a separate subscription mechanism.

The register/resolve pattern ($\downarrow DEFS$ to register, ρ to resolve) is inversion of control via ρ. Every value in the REST representation is a ρ-application, including the serialization format. JSON keys are role names (constants in compiled schemas). HTTP status codes are condition results over the violation set. Media types are platform constants registered in *DEFS* by the runtime. No value is produced outside ρ.

6 Example

An order domain is declared in FORML 2:

```
Order(.OrderId) is an entity type.
Customer(.Name) is an entity type.
Order was placed by Customer.
```

```
  Each Order was placed by exactly one
  Customer.
Status 'In Cart' is initial in
  State Machine Definition 'Order'.
Transition 'place' is from
  Status 'In Cart'.
Transition 'place' is to
  Status 'Placed'.
Transition 'ship' is from
  Status 'Placed'.
Transition 'ship' is to
  Status 'Shipped'.
```

The constraint reading compiles to a restriction (6). ρ yields the following.

$$(\rho\, reading) : P_{placedBy} = Filter(count(s_1) > 1) : P_{placedBy}$$

The result is the set of Orders with more than one Customer in *placedBy*. An empty set means the constraint is satisfied.

`POST /orders {"customer":"acme"}` is the application $SYSTEM : x$ where the addressed entity is *orders* and the operation is *create* (12). By (11), $(\rho\, orders) : create$ applies the pipeline:

$$P' = P \cup \{Order(\texttt{ord-1}),\ placedBy(\texttt{ord-1}, \texttt{acme}),\ SM(\texttt{ord-1}, \texttt{In Cart})\}$$

The response includes links derived from P' (Theorem 4).

```
{ "id": "ord-1", "customer": "acme",
  "status": "In Cart",
  "_links": {
    "place": {
      "href": "/orders/ord-1/transition",
      "method": "POST" } } }
```

Following "place" applies (13). The new representation contains $\{ship\}$, not $\{place\}$, because the status is now Placed and only the outgoing transition from Placed is available. If a second command assigns a different Customer to the same Order, the UC restriction produces a non-empty violation set and the command is rejected. The violation is returned as the original reading (Corollary 2).

7 Execution Semantics

7.1 Evaluation

μ evaluates by applicative-order β-reduction [1, Sec. 13.3.1]. It repeatedly replaces the innermost application $(x : y)$ with $(\rho\, x) : y$, where ρ resolves x to a λ-term under definitions D [1, Sec. 12.9]. All evaluation is over finite P with monotonic derivations. Reduction terminates for all requests.

7.2 Evaluation Strategy

Three constraints govern execution:

1. **Reduction order:** applicative (call-by-value). All arguments are reduced before the function is applied.
2. **Scope:** evaluation is confined to the request-local partition of D. RMAP [5, Ch. 17] determines the partition from the schema's uniqueness constraints.
3. **Derivation:** forward chaining, monotonic, evaluated to the least fixed point per request. Since *derive* only adds facts over finite P_{local}, the fixed point exists (Knaster-Tarski) and is reached in $\leq |P_{local,max}|$ iterations. The derivation chain (which rules fired, in what order, producing which facts) is recorded and available on demand as part of the representation.

The completeness result (Theorem 3) assumes evaluation to the least fixed point under this strategy.

7.3 Boundary Conditions

The system operates over facts (P), computations (O via ρ), and inputs (I), the operand x in $\mu(SYSTEM : x)$ [1, Sec. 14.3.1]. Time is an input fact in x, not hidden state. Effects are directives in the output o. Their outcomes are facts in D'. Identity follows from the reference scheme [5, Ch. 5]. *resolve* in (12) applies it. The world assumption per noun [5, Ch. 7] governs incompleteness. Under CWA, a fact not in P is false. Under OWA, it is unknown. Derivation rules for function invocation operate under OWA because the rules capture known reasons to act, not all possible reasons. When the derived course of action is *unknown*, that is itself a reason to escalate to a human or broader-context agent.

Remark (World assumption on populating functions). The theorems are agnostic to the population source. All populating functions are runtime functions registered in *DEFS*. The world assumption determines the interpretation of absence. A function that exhaustively enumerates its domain operates under CWA: if no matching value is found, the fact is definitively absent. A function that cannot exhaustively enumerate its domain operates under OWA: absence of the fact means unknown, not false. Deontic constraints evaluated over OWA-populated facts are sound but not complete. A reported violation is genuine. The absence of a violation is not a guarantee.

7.4 Isolation

A state machine (13) is a left fold over an event stream. Each event is an input x. The accumulator is the state D. The fold body is $\mu(SYSTEM : x) = \langle o, D' \rangle$. The application is this fold. The fold is well-defined only if each μ runs to completion before the next event mutates D. Otherwise two concurrent applications of μ may read the same D, and one silently overwrites the other.

Backus states this directly: "the state cannot change during the computation of system:x" [1, Sec. 14.6(c)]. We

make the requirement a definition so that theorems can reference it.

Definition 2 (Cell Isolation)**.** For each cell $\langle CELL, n, c\rangle$ in D, at most one μ application that writes to n may be in progress at any time. Concurrent μ applications over disjoint cells are permitted.

RMAP [5, Ch. 17] determines which facts belong to which cell from the schema's uniqueness constraints. Each entity is a cell. Its fold is independent of other entities' folds. A single-threaded runtime (e.g. a WASM isolate) satisfies Definition 2 trivially. A distributed system requires one writer per cell (actor mailbox, partitioned queue, or single-writer lease).

Theorems 3–5 hold under Definition 2. Without it, Theorem 3 fails silently. Two concurrent *create* operations on the same cell may both pass *validate* against stale P and produce a D' that violates a uniqueness constraint. Unlike a grammar violation (Theorem 1), which is caught at parse time, a cell isolation violation produces inconsistent state without an error.

7.5 Distributed Evaluation

Each cell from Sec. 4.2 stores the 3NF row, including the state machine's current status as one of its facts. The event stream E is demultiplexed into per-cell streams by RMAP [5, Ch. 17]:

$$E_n = \mathit{Filter}(\mathit{eq} \circ [\mathit{RMAP}, \bar{n}]) : E \tag{15}$$

Each cell folds its stream independently:

$$D'_n = \mathit{foldl}\ \mu_n\ D_n\ E_n \tag{16}$$

where μ_n is SYSTEM restricted to cell n. By Definition 2, no μ_n reads another cell's in-progress state. Cross-cell queries read committed state:

$$P = \bigcup_n \uparrow \mathit{FILE} : D_n \tag{17}$$

The router distributes events via $\alpha\,\mathit{RMAP}$ (apply-to-all), and each cell reduces its partition via $\mathit{foldl}\ \mu_n$. Adding cells to the cluster scales SYSTEM horizontally without changing its definition.

A browser runtime is a node in the same model. Cells replicated locally fold events with zero latency. The event stream syncs to the server asynchronously. Because the fold is pure and deterministic (13), two nodes folding the same events reach the same state. Constraints that depend only on the local cell (cardinality, value) evaluate immediately. Constraints that span cells (subset, external uniqueness) evaluate against committed state (17) after sync. Local cells are untrusted: the server fold is authoritative, and alethic violations on the server reject the event regardless of the local evaluation. Local constraint evaluation is optimistic feedback, not a security boundary.

For two peers sharing a cell, one proposes an event and the other runs *validate* (12) against $P \cup \mathit{resolve}(S, e)$. The event is accepted iff V contains no alethic violation. The constraint set C_S is the consensus predicate (Corollary 4). For $n > 2$ peers, a shared event log with total ordering is required. Each event references the hash of its predecessor, forming a chain that guarantees integrity and deterministic replay. Peers compare chain tips and adopt the longest chain on fork. The fold (16) over the shared log produces the same state on every peer.

For anonymous peers, three additional properties are required. First, each event carries a cryptographic signature identifying the peer that appended it. Second, reading events ($\mathit{compile} \circ \mathit{parse}$ applications that modify *DEFS*) require authorization. This is a deontic constraint evaluated on every fold: a peer not authorized to modify *DEFS* can append fact events but not reading events. Third, every peer validates independently on fold, so a malicious event that violates an alethic constraint is detected deterministically by every honest peer. No peer trusts another peer's evaluation. The hash chain provides ordering and tamper detection, the constraints provide validity, the signatures provide identity, and the authorization constraints provide permission. These four properties compose from the existing formalism without a separate security layer.

8 Properties

The five theorems form a chain. Each depends on the previous one and eliminates one source of failure in the path from domain knowledge to running software.

Theorem 1 (Grammar Unambiguity)**.** Let N be a declared set of object type names and F a declared set of predicate readings, such that no element of N contains a formal item of the grammar as a substring. Then each FORML 2 sentence over (N, F) has exactly one parse.

Proof. FORML 2 sentences fall into three structural families, each determined by its prefix:

(a) Quantified constraints. Structure: $[\mathit{modal}]\ \mathit{quant}_1\ n_1\ r\ \mathit{quant}_2\ n_2\ [\mathit{qual}]$ where $\mathit{modal} \in \{$"It is obligatory that", "It is forbidden that", "It is permitted that", "It is impossible that", "It is possible that", $\varepsilon\}$; $\mathit{quant}_1 \in \{$"Each", "For each", "the same", "No", $\varepsilon\}$; $\mathit{quant}_2 \in \{$"at most one", "some", "exactly one", "at least k and at most m", "more than one", "no", $\varepsilon\}$. Noun positions $n_i \in N$ are resolved by longest-first matching. The predicate $r \in F$ is the maximal token sequence between noun positions. Each structural position admits exactly one parse because (i) modal prefixes are pairwise non-overlapping, (ii) quantifier phrases are pairwise distinct, (iii) no $n \in N$ contains a formal item as a substring (by hypothesis), and (iv) longest-first matching is deterministic. The constraint kind is determined by $(\mathit{quant}_1, \mathit{quant}_2)$: (Each, at most one) $\to$ UC; (Each,

some) $\to$ MC; (Each, exactly one) $\to$ UC+MC; (Each, at least $k\ldots$) $\to$ FC; (the same, more than one) $\to$ UC$^-$; (No, ε) $\to$ XC. This family also generalizes to n-ary predicates $(n_1 \ldots n_k)$ by the same longest-first procedure.

(b) Conditional constraints. Structure: "If" *antecedent* "then" *consequent*
The "If... then" frame is unambiguous (unique split on "then"). Ring constraints (IR, AS, SY, AT, IT, TR, AC, RF) are identified by all noun bases being the same type. The kind is determined by polarity ("it is impossible that" in the consequent) and arity ("and" in the antecedent). Subset constraints (SS) are identified by existential "some" in the antecedent and anaphoric "that" in the consequent with mixed noun types. Derivation rules occupy the remainder.

(c) Multi-clause constraints. Keyword-delimited: "if and only if" (EQ), "exactly one of the following holds" (XO), "at most one of the following holds" (XC), "at least one of the following holds" (OR). The keyword is unambiguous. Clauses are separated by ";" or "or".

The three families are distinguished by their sentence prefix (quantifier word, "If", or keyword), which is determined at position (1). Within each family, the parse is deterministic by the arguments above. The reference implementation [8] constructively demonstrates unambiguous round-trip parsing for all 17 constraint kinds. □

The hypothesis on N is enforced by the compiler, which rejects any declared name containing a formal item of the grammar as a substring.

Theorem 2 (Specification Equivalence)**.** Let $\mathit{parse} : R \to \Phi$ and $\mathit{compile} : \Phi \to O$ be injective, with R the set of FORML 2 readings, Φ the set of FOL formulas, and O the set of FFP objects. Then:

$$\mathit{parse}^{-1} \circ \mathit{compile}^{-1} \circ \mathit{compile} \circ \mathit{parse} = \mathit{id}_R \tag{18}$$

and $\rho(\mathit{compile}(\mathit{parse}(r)))$ is the executable for every $r \in R$.

Proof. *parse* is injective by Theorem 1: each sentence has one parse, so distinct sentences yield distinct formulas. *compile* is injective: each constraint kind compiles to a restriction (6) with a predicate from one of two families: cardinality predicates parameterized by bounds $[k, m]$, and membership predicates parameterized by target set and polarity. Distinct kinds produce distinct parameter tuples, and thus distinct FFP objects. Both *parse* and *compile* have left inverses ($\mathit{compile}^{-1}$ is Halpin and Curland's verbalization algorithm [6]); composing: $\mathit{parse}^{-1} \circ \mathit{compile}^{-1} \circ \mathit{compile} \circ \mathit{parse} = \mathit{id}_R$. Executability: ρ maps the compiled object to a function by [1, Sec. 13.4]. □

Remark (Surjectivity). *compile* is not surjective. There exist FFP objects in O that no FORML 2 reading produces. This is intentional: runtime-registered functions (Sec. 5.3) occupy the complement. FORML 2 readings cover the domain layer. Registered functions cover the platform layer. Together they span the objects in *DEFS*.

Theorem 3 (Completeness of State Transfer)**.** Let $F_S : 2^P \to 2^P$ be the monotonic operator that applies all derivation rules in schema S to a population, and let C_S be the set of constraint objects in S. Under Definition 2, *create* (12) applied to input I and state D produces $\langle o, D' \rangle$ such that:

1. $P' = P \cup \mathit{resolve}(S, I)$ contains all entity facts entailed by (S, I);
2. $P'' = \mathit{lfp}(F_S, P')$, the least fixed point of F_S above P';
3. $V = \bigcup\{(\rho\, c) : P'' \mid c \in C_S\}$ collects all violations (each constraint produces a possibly empty set via restriction);
4. o is the REST representation of $(P'', V, \mathit{links}(s))$ and $D' = D[\mathit{FILE} \mapsto P'']$ if V contains no alethic violation, else $D' = D$.

Proof. P is finite. F_S is monotonic (derivation rules only add facts). By Knaster-Tarski, $\mathit{lfp}(F_S, P')$ exists and is unique. Iterative application of F_S reaches it in at most $|P''_{\max}| - |P'|$ steps. *validate* applies every $c \in C_S$ as a restriction over the fixed point; each produces a (possibly empty) set of violations. *emit* constructs o from P'', V, and $\mathit{links}(s)$ (Theorem 4). The state does not change during evaluation [1, Sec. 14.6(c)]; Definition 2 ensures no concurrent write to the same cell. □

Theorem 4 (HATEOAS as Projection)**.** All links in the representation are θ_1 operations on P and S.

(a) Transition links. Let $T = \{(s_i, s_j, e_k)\} \subseteq P$ be transition facts and s the current status. Then:

$$\mathit{links}(s) = \pi_{\mathit{event}}(\mathit{Filter}(p) : T) \tag{19}$$

where $p(t) = (s_{\mathit{from}}(t) = s) \vee \mathit{anc}(s_{\mathit{from}}(t), s)$ and $\mathit{anc}(s', s)$ holds when s is reachable from s' through the transitive closure of the transition graph (computed with a visited set to handle cycles). Transition graphs may contain cycles (e.g. a support request moving between investigating and waiting states). It is obligatory that each cycle in the transition graph has some exit Transition. This deontic mandatory constraint ensures liveness without requiring structural acyclicity.

(b) Navigation links. Let F be the set of all fact types in S and let U_f denote the role set spanned by the UC on fact type f. Let $S_f = R_f \setminus U_f$ be the non-spanned roles. For a noun n participating in f, the navigation algorithm is:

1. **Related collection.** For every f in which n participates, f is a related collection on n, filtered by n. This always applies.
2. **Parent/child.** If $S_f \neq \emptyset$, each noun playing a role in S_f is a child of each noun playing a role in U_f. The

UC establishes a functional dependency from U_f to S_f.

3. **Peer resource.** If $S_f = \emptyset$ (spanning UC), f appears as a navigable peer resource at the level of its highest participant in the navigation hierarchy.

Unary fact types ($|R_f| = 1$) are boolean properties with no navigation target. Binary fact types ($|R_f| = 2$) with a single-role UC reduce to the standard parent/child case where the constrained role's noun is the child. For the binary case, the navigation links for an entity e of noun n are:

$$children(n) = \pi_{noun(r_c)}(Filter(uc(r_c) \wedge eq \circ [noun(\bar{r}_c), \bar{n}]) : F_B) \tag{20}$$

$$parent(n) = \pi_{noun(\bar{r}_c)}(Filter(uc(r_c) \wedge eq \circ [noun(r_c), \bar{n}]) : F_B) \tag{21}$$

$$nav(e, n) = children(n) \cup parent(n) \cup related(n) \cup peers(n) \tag{22}$$

where $related(n)$ collects all fact types in which n participates as related collections, and $peers(n)$ collects spanning-UC fact types at the appropriate hierarchy level. The full link set combines both:

$$links_{full}(e, n, s) = nav(e, n) \cup links(s) \tag{23}$$

Proof. (a) $T \subseteq P$. *Filter* is restriction (6); π is projection (4). Both are θ_1 operations. *(b)* $F \subseteq S$. Each component of *nav* is a projection (4) over a restriction (6). *Filter* (2) is derived from α and condition. The predicate composes *eq* with selector and constant, all primitives from Table 1. For the parent/child case, the UC on U_f establishes a functional dependency from U_f to S_f (RMAP step 2). Related collections are restrictions of f filtered by the participant noun. Peer resources are fact types whose spanning UC produces no functional dependency, presented at the hierarchy level of their highest participant. The navigation algorithm is a single pass over fact types, computing S_f for each and emitting links accordingly. All components of $links_{full}$ are θ_1 operations on S and P. □

Corollary 1 (Deletion as Terminal State)**.** Let s_{del} be a state with no outgoing transitions in T, i.e. $\neg\exists\, t \in T : s_{from}(t) = s_{del}$. Then $links(s_{del}) = \emptyset$ by Theorem 4, and the entity is excluded from query results by $Filter(p_{live}) : P$ where $p_{live}(e) = \exists\, t \in T : s_{from}(t) = status(e)$.

Theorem 5 (Derivability)**.** Define the *representation* of an entity e in population P under schema S as:

$$\begin{aligned} repr(e, P, S) = &\{(\rho\, s_i) : f \mid f \in P,\ f \text{ involves } e\} \\ &\cup \{(\rho\, r) : P \mid r \in S_{derivations}\} \\ &\cup \{(\rho\, c) : P \mid c \in S_{constraints}\} \\ &\cup links_{full}(e, n, status(e, P)) \end{aligned}$$

where $status(e, P)$ is the current status of e, a fact in P (absorbed into the entity's cell by RMAP), and $links_{full}$ combines navigation links (Theorem 4) with transition links (Theorem 4). Then for every $v \in repr(e, P, S)$, there exists $f \in O$ such that $v = (\rho\, f) : P$.

Proof. Each component of *repr* is a ρ-application by construction. Selectors extract attributes. Derivation rules are FFP objects whose ρ-image computes derived facts. Constraints are restrictions (6) whose ρ-image produces a set of violations (empty if satisfied). Links are θ_1 operations (Theorem 4). No value in *repr* is produced by a mechanism outside ρ. □

Corollary 2 (Violation Verbalization)**.** Let $c \in C_S$ be a constraint object compiled from reading $r \in R$. If $(\rho\, c) : P_B \neq \emptyset$, the violation output is $compile^{-1}(c)$, recovering r by Theorem 2. The error message is the original reading.

Each θ_1 primitive has a FORML 2 verbalization. Projection verbalizes as role selection, join as qualified composition, restriction as constraint predicate, tie as self-reference. Halpin and Curland [6] give the automated verbalization algorithm for ORM 2 constraints. Composed θ_1 operations verbalize via FORML 2 qualified readings ("who...", "that..."), which parse unambiguously by Theorem 1.

Corollary 3 (Closure Under Self-Modification)**.** Let $D' = D$ after applying *SYSTEM* with operation $compile \circ parse$ to a set of FORML 2 readings R'. Then Theorems 1–5 hold for all subsequent applications of *SYSTEM* over D'.

Proof. Each new reading $r' \in R'$ parses unambiguously (Theorem 1 depends on the grammar, not on D). The compiled objects are faithful to the readings (Theorem 2 depends on the injectivity of *parse* and *compile*, which are stateless). The new constraints and derivation rules enter *DEFS* via $\downarrow DEFS$, joining the existing definitions. Theorem 3 evaluates all $c \in C_S$, including the new ones. Theorems 4 and 5 operate over P and S, both of which now include the new facts and schemas. No theorem's proof depends on D being fixed at initialization. □

Corollary 4 (Constraint Consensus)**.** For two peers A and B sharing population P, if A proposes event e and B applies *validate* (12) to $P \cup resolve(S, e)$, the event is accepted iff V contains no alethic violation. The constraint set C_S is the consensus predicate. No external consensus protocol is required.

For $n = 2$, the constraints are sufficient for consensus. For $n > 2$, a total ordering over the shared event log is additionally required so that all peers fold (16) in the same order. Safety follows from the formalism: every peer folds the same totally-ordered log with the same deterministic *SYSTEM*, producing the same state. Alethic violations

are detected independently by every honest peer. Liveness is provided by the single-writer cell. Each cell (Definition 2) has at most one writer at any time. The platform guarantees this by routing all writes to one instance and electing a new instance on failure (the same mechanism as a single-leader append-only log with leader election under partial synchrony). For the peer-to-peer extension, the shared event log with hash-chained entries is the equivalent of a log partition. Liveness requires leader election via majority vote, the standard result from partially synchronous consensus.

Common operational concerns map directly to the formalism: versioning is the event stream (E), migration is ingestion of new readings (Corollary 3), backward compatibility is preserved by construction (no theorem depends on D at initialization), and auditability is the derivation trace (Sec. 7.2).

The algebraic laws of FP [1, Sec. 12.2] are equations between functional forms. Since FFP objects represent the same forms via ρ [1, Sec. 13.3.2], an FFP object can be transformed to an equivalent object by applying these laws. Ahead-of-time compilation from objects to optimized evaluation trees is one such transformation.

9 Middleware Elimination

Traditional middleware (authentication, validation, rate limiting, transformation) is either a restriction over P, a COMP derivation, or a fact in P. Authorization, for instance, is a derivation. "User accesses Domain iff User belongs to Organization that manages Domain" compiles to $\langle COMP, f, g\rangle$ whose ρ-image computes the access predicate over P. External service calls, database persistence, and UI rendering are not middleware either. They are runtime functions registered into *DEFS* and applied via ρ (Sec. 5.3). Theorem 5 leaves no room for a computation layer outside ρ. Every value in the representation is a ρ-application.

10 Conclusion

AREST is a Backus AST system [1] whose definitions are compiled FORML 2 readings [5] and whose state is Codd's named set [2] of ORM 2 elementary facts. REST endpoints [4] are applications of the system function. Ingesting readings is self-modification. Transitions produce hypermedia. The five theorems form a chain: the grammar is unambiguous (Theorem 1), so specifications compile faithfully to executables (Theorem 2), so state transfer is complete (Theorem 3), so all links (transitions and entity hierarchy) are projections of the population (Theorem 4), so every value in the representation is a ρ-application (Theorem 5). Self-modification preserves all five (Corollary 3). The system grows by ingesting facts and never loses its guarantees.

The five properties compose into a guarantee for automated agents, addressing two distinct sources of LLM hallucination. First, the specification-execution gap. An LLM writing FORML 2 readings produces unambiguous specifications (Theorem 1) that are the executables (Theorem 2). An LLM consuming the API reads values that are ρ-applications over P (Theorem 5) and discovers every valid action via HATEOAS links (Theorem 4). There are no undocumented endpoints, no hidden middleware, no informal contracts to hallucinate through. Second, the reasoning chain gap. LLMs do not reliably chain logical inferences, and errors compound over steps. The forward chainer in *derive* (12) evaluates derivation rules to the least fixed point (Theorem 3), and the derived facts appear in the representation (Theorem 5). The LLM reads *conclusions*, not premises. The logical chain that produced them was evaluated deterministically by the engine, outside the LLM's context. The LLM does not reason through the chain; it consumes the result. When the chain itself is needed (for debugging, auditing, or explanation), the derivation trace is available on demand as a sequence of rule applications over P, each a ρ-application (Theorem 5).

The gap between domain knowledge and running software is the gap where requirements drift, implementations diverge, and production breaks. AREST eliminates the gap by identifying what was never separate to begin with.

Acknowledgments

Claude (Anthropic) was used as a programming assistant during implementation and as a drafting aid during manuscript preparation. All architectural decisions and the composition of FFP/AST with REST were made by the human author.

References

[1] J. Backus, "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs," *Commun. ACM*, 21(8):613–641, 1978.

[2] E.F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Commun. ACM*, 13(6):377–387, 1970.

[3] E.F. Codd, "Relational Completeness of Data Base Sublanguages," in R. Rustin, ed., *Data Base Systems*, Prentice-Hall, pp. 65–98, 1972.

[4] R.T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[5] T. Halpin, *Information Modeling and Relational Databases*, 2nd ed. Morgan Kaufmann, 2008.

[6] T. Halpin and M. Curland, "Automated Verbalization for ORM 2," in *OTM Workshops*, pp. 1181–1190, 2006.

[7] T. Halpin, M. Curland, and CS445 Class, "ORM 2 Constraint Verbalization: Part 1," Technical Report ORM2-02, Neumont University, 2006.

[8] T. Halpin and M. Curland, NORMA (Natural Object-Role Modeling Architect), open-source reference implementation, `https://github.com/ormsolutions/NORMA`.